TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

LABORATORY REPORT

Lab 13: Semaphores in Your Own Kernel

Operating Systems

Student: Gafenco Victor

Group: FAF-241

Instructor: Assistant lecturer, Patricia Reițman

Chișinău - 2026

CONTENTS

Abstract **3**

Introduction **4**

1 Part 1: The numbers and the user side **5**

1.1 System call numbers, prototypes, stubs, and test programs 5

1.2 Check 1: The calls reach the kernel 7

2 Part 2: The kernel side **8**

2.1 Implementation of `kernel/sem.c` and syscall dispatch 8

2.2 Check 2: `semtest` 9

3 Part 3: A semaphore as a lock: `mix` **11**

3.1 Running `mix` without and with a lock 11

3.2 Observations for Part 3 11

Conclusions **13**

ABSTRACT

The goal of this laboratory is to implement counting semaphores directly inside the xv6 RISC-V operating system kernel as three new system calls—`sem_init` (24), `sem_down` (25), and `sem_up` (26)—together with the optional non-blocking `sem_trydown` (27). In Part 1, the system call numbers, user-space prototypes, `usys.pl` trap stubs, and two test utilities (`user/semtest.c` and `user/mix.c`) are wired into the build and verified to reach the kernel trap handler (`unknown sys call 24`). In Part 2, `kernel/sem.c` implements a shared table of eight semaphores (`sems[NSEM]`), each consisting of an integer counter protected by a spinlock and synchronized via xv6's two-step sleep mechanism (`sleep_prepare(s)`, `release(&s->lock)`, `sleep()`) and `wakeup(s)`, passing both the parent-before-child ordering test and the three-round ping-pong test in `semtest`. In Part 3, running `mix` without and with semaphore 3 as a binary lock demonstrates character-level interleaving across 31 `write()` system calls per line on a 3-CPU RISC-V machine versus atomic whole-line output under `mix lock`.

Keywords: xv6, RISC-V, kernel semaphores, system calls, spinlock, sleep_prepare, sleep, wakeup, critical section

INTRODUCTION

Goal of the laboratory (Scopul lucrării)

Whereas Labs 11 and 12 used POSIX semaphores from the C library on Linux from the outside, the purpose of this laboratory work is to build counting semaphores inside the xv6 RISC-V kernel itself as new system calls (`sem_init`, `sem_down`, `sem_up`, plus optional `sem_trydown`). Specifically, the scope and objectives of the laboratory are:

1. **Part 1 (The numbers and the user side):** Assign system call numbers `24` (`SYS_sem_init`), `25` (`SYS_sem_down`), and `26` (`SYS_sem_up`) in `kernel/syscall.h`, declare their prototypes in `user/user.h`, add their `ecall` stubs in `user/usys.pl`, create `user/semtest.c` and `user/mix.c`, add them to `UPROGS` in `Makefile`, and verify Check 1 (`unknown sys call 24` and `25`, with the child printing before the parent).
2. **Part 2 (The kernel side):** Implement `kernel/sem.c` with a table of `NSEM = 8` kernel semaphores (`struct sem { struct spinlock lock; int value; }`) and handlers `sys_sem_init`, `sys_sem_down`, and `sys_sem_up` using `sleep_prepare(s)` and `sleep()` / `wakeup(s)`, link `$K/sem.o` in `Makefile`, register the handlers in `kernel/syscall.c`, and pass Check 2 (`semtest: OK`).
3. **Part 3 (A semaphore as a lock: `mix`):** Run `mix` without a lock and with semaphore 3 as a lock (`mix lock`) across 3 virtual CPUs, and explain why characters interleave without the lock, which lines form the critical section, why `sem_down` tests `s->value == 0` in a `while` loop, and why `s->lock` must be released before `sleep()`.
4. **Optional (`sem_trydown`):** Add system call `27` (`SYS_sem_trydown`), which decrements the semaphore if positive or returns `-1` immediately when the value is `0`, and verify it with `user/trydowntest`.

All kernel and user changes were built and tested in `/home/victor/xv6-riscv` on QEMU (`-smp 3`). The complete patch is provided in `xv6-sem.patch` and answers are recorded in `notes.txt`.

1 PART 1: THE NUMBERS AND THE USER SIDE

1.1 System call numbers, prototypes, stubs, and test programs

Six user-side and header additions expose the semaphore calls to user programs:

1. **`kernel/syscall.h`:** Assign numbers 24, 25, 26 (and 27 for the optional `sem_trydown`) after `SYS_sync` (22) and `SYS_trace` (23):

```
#define SYS_sync        22
#define SYS_trace       23
#define SYS_sem_init    24
#define SYS_sem_down    25
#define SYS_sem_up      26
#define SYS_sem_trydown 27
```

2. **`user/user.h`:** Declare the user-space system call prototypes:

```
int sync(void);
int trace(int);
int sem_init(int, int); // new: id, start value
int sem_down(int);      // new: take one, or sleep
int sem_up(int);        // new: give one back
int sem_trydown(int);   // optional: take one, or return -1 at once
```

3. **`user/usys.pl`:** Add stub entries so `usys.pl` generates the RISC-V `li a7, SYS_sem_*; ecall; ret` assembly routines in `user/usys.S`:

```
entry("sync");
entry("trace");
entry("sem_init");
entry("sem_down");
entry("sem_up");
entry("sem_trydown");
```

4. **`user/semtest.c`:** Test 1 checks parent-before-child ordering using semaphore 0 initialized to 0 (while the parent pauses for 10 ticks), and Test 2 runs a 3-round ping-pong alternation using semaphore 1 (started at 1) and semaphore 2 (started at 0):

```
// semtest.c: two tests for the kernel semaphores (Lab 12).
#include "kernel/types.h"
#include "user/user.h"

int
main(void)
{
  // Test 1: order. Semaphore 0 starts at 0, so the child must wait.
  sem_init(0, 0);
  if (fork() == 0) {
    sem_down(0);
    printf("child: I ran after the parent\n");
    exit(0);
  }
  pause(10);
  printf("parent: done, I let the child go\n");
  sem_up(0);
```

```
  wait(0);

  // Test 2: ping-pong. Semaphore 1 = ping's turn, semaphore 2 = pong's turn.
  sem_init(1, 1);
  sem_init(2, 0);
  if (fork() == 0) {
    for (int i = 1; i <= 3; i++) {
      sem_down(2);
      printf("  pong %d\n", i);
      sem_up(1);
    }
    exit(0);
  }
  for (int i = 1; i <= 3; i++) {
    sem_down(1);
    printf("ping %d\n", i);
    sem_up(2);
  }
  wait(0);
  printf("semtest: OK\n");
  exit(0);
}
```

5. **`user/mix.c`:** Two processes each print five 30-letter lines (`A`s in the parent, `b`s in the child), issuing one `write(1, p, 1)` system call per character, optionally guarded by semaphore 3:

```
// mix.c: two processes print five lines each, one letter per write().
//   mix        no lock: the letters of the two processes can mix
//   mix lock   semaphore 3 used as a lock: every line stays whole
#include "kernel/types.h"
#include "user/user.h"

void
lines(char *s, int uselock)
{
  for (int k = 0; k < 5; k++) {
    if (uselock)
      sem_down(3);
    for (char *p = s; *p; p++)
      write(1, p, 1);
    if (uselock)
      sem_up(3);
  }
}

int
main(int argc, char *argv[])
{
  int uselock = (argc > 1);
  sem_init(3, 1);
  if (fork() == 0) {
    lines("bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb\n", uselock);
    exit(0);
  }
  lines("AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA\n", uselock);
  wait(0);
  exit(0);
}
```

6. **`Makefile`:** Add `$U/_semtest\`, `$U/_mix\`, and `$U/_trydowntest\` after `$U/_zombie\` in

UPROGS.

1.2 Check 1: The calls reach the kernel

Running `semtest` before adding the kernel side produces:

```
$ semtest
3 semtest: unknown sys call 24
4 semtest: unknown sys call 25
child: I ran after the parent
parent: done, I let the child go
```

Without kernel semaphores, `sem_init(0, 0)` (syscall 24) and `sem_down(0)` (syscall 25) fail immediately with `-1`, so the child does not sleep during the parent's `pause(10)` and prints `child: I ran after the parent` *before* the parent prints `parent: done, I let the child go`.

2 PART 2: THE KERNEL SIDE

2.1 Implementation of `kernel/sem.c` and syscall dispatch

The file `kernel/sem.c` defines a static table of `NSEM = 8` semaphores shared across all processes. Each entry pairs a spinlock (`lock`) with an integer counter (`value`). In our xv6 tree, sleeping uses the two-step sequence `sleep_prepare(s)`, `release(&s->lock)`, `sleep()`:

```
// sem.c: counting semaphores inside the xv6 kernel (Lab 12).
// A small table of NSEM semaphores, shared by every process.
// Each one is a number plus a spinlock that protects the number.

#include "types.h"
#include "riscv.h"
#include "defs.h"
#include "param.h"
#include "spinlock.h"
#include "proc.h"

#define NSEM 8

struct sem {
  struct spinlock lock;   // a zeroed spinlock is a free spinlock
  int value;              // how many may still pass
};

static struct sem sems[NSEM];

// sem_init(id, value): give semaphore id its start value.
uint64
sys_sem_init(void)
{
  int id, value;
  argint(0, &id);
  argint(1, &value);
  if (id < 0 || id >= NSEM || value < 0)
    return -1;
  acquire(&sems[id].lock);
  sems[id].value = value;
  release(&sems[id].lock);
  return 0;
}

// sem_down(id): take one unit; if there is none, sleep until there is.
uint64
sys_sem_down(void)
{
  int id;
  argint(0, &id);
  if (id < 0 || id >= NSEM)
    return -1;
  struct sem *s = &sems[id];

  acquire(&s->lock);
  while (s->value == 0) {
    sleep_prepare(s);     // I will sleep on the address of s
```

```
    release(&s->lock);    // never sleep holding a spinlock
    sleep();              // returns at once if sem_up came in between
    acquire(&s->lock);
    if (killed(myproc())) { // kill must still work on a waiting process
      release(&s->lock);
      return -1;
    }
  }
  s->value--;
  release(&s->lock);
  return 0;
}

// sem_up(id): give one unit back and wake whoever sleeps on s.
uint64
sys_sem_up(void)
{
  int id;
  argint(0, &id);
  if (id < 0 || id >= NSEM)
    return -1;
  struct sem *s = &sems[id];

  acquire(&s->lock);
  s->value++;
  wakeup(s);
  release(&s->lock);
  return 0;
}

// sem_trydown(id): take one unit if available, or return -1 at once.
uint64
sys_sem_trydown(void)
{
  int id;
  argint(0, &id);
  if (id < 0 || id >= NSEM)
    return -1;
  struct sem *s = &sems[id];

  acquire(&s->lock);
  if (s->value == 0) {
    release(&s->lock);
    return -1;
  }
  s->value--;
  release(&s->lock);
  return 0;
}
```

In `Makefile`, `$K/sem.o \` is added after `$K/sysproc.o \` in `OBJS`. In `kernel/syscall.c`, `sys_sem_init`, `sys_sem_down`, `sys_sem_up`, and `sys_sem_trydown` are declared `extern` and added to `syscalls[]` (and `syscallnames[]`).

2.2 Check 2: `semtest`

Booting xv6 with `make qemu` and running `semtest`:

```
$ semtest
parent: done, I let the child go
child: I ran after the parent
ping 1
  pong 1
ping 2
  pong 2
ping 3
  pong 3
semtest: OK
```

Even though the parent pauses for 10 ticks before printing, the child blocks in `sem_down(0)` and prints second, after the parent calls `sem_up(0)`. Then `ping` and `pong` alternate strictly for three rounds before printing `semtest: OK`.

3 PART 3: A SEMAPHORE AS A LOCK: `MIX`

3.1 Running `mix` without and with a lock

Running `mix` twice without arguments (no lock) in QEMU (`-smp 3`):

```
$ mix
AAAbAbbbbbbbbbbAbAbAbAbAbAbAbAbAbAAbAbAbbbbbbAAbAb
AbbAbAbAbAbAbAbAbAb
bAbAbAbAbAbAbAAbAbAbAbAAAAAAbbbbbbbb
bbbAbAbbbAAAAAAbbAbbAAbbbbbbbAbAb
bbbbbbbbbbA
AbAbAAbAAbAbAAbAbbAbAAAbAAbbbAbAbAbAbAbAbAbAbAbAbAbAbAb
bAbAbAbA
AbAbAbAbbbbbbbbbbbbbbAbAbAbAbAbAbAbAAbAAAbAbAbAbAbAbA
AAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

Running `mix lock` (using semaphore 3, initialized to `1`, as a binary lock around each line):

```
$ mix lock
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
```

Testing the optional non-blocking `sem_trydown` (system call 27) with `trydowntest`:

```
$ trydowntest
first sem_trydown(4): 0 (expected 0)
second sem_trydown(4): -1 (expected -1)
after sem_up, sem_trydown(4): 0 (expected 0)
```

3.2 Observations for Part 3

1. **Without the lock, why do the letters mix? (How many system calls does one line cost?)** Each line of `mix` contains 30 letters plus 1 newline character (`'\n'`), and the inner loop `for (char *p = s; *p; p++) write(1, p, 1);` invokes a separate `write()` system call for every single character—so **one line costs 31 `write()` system calls**. Because QEMU boots xv6 with 3 virtual CPUs (`-smp 3`), the parent and child processes run truly in parallel on separate CPU cores (and can also be preempted by timer interrupts between calls). While the console driver serializes each individual 1-byte `write()`, nothing keeps the 31 `write()` calls of a line together without a semaphore lock, so characters from both processes interleave.
2. **With the lock every line is whole. Which two lines of `mix.c` make one line a critical section?**

The call `sem_down(3);` right before the character loop `for (char *p = s; *p; p++) write(1, p, 1);` and the call `sem_up(3);` right after that loop (executed when `uselock` is true). Because semaphore 3 starts at `1`, whichever process calls `sem_down(3)` first decrements it to `0` and prints all 31 characters of its line before calling `sem_up(3)`, while the other process sleeps in `sem_down(3)`.

3. **In `sem_down` the test is `"while (s->value == 0)"`, not `"if"`. Why can a process wake up and still find 0?** In `sys_sem_up()`, `wakeup(s)` wakes up **all** processes sleeping on channel `s` (broadcast semantics), and another running process on another CPU could also call `sem_down` before a woken process runs. When `sem_up` increments `s->value` from `0` to `1`, the first process to acquire `s->lock` exits the loop and decrements `s->value` back to `0`. Any other woken process that acquires `s->lock` afterward finds `s->value == 0` again and must loop back to sleep via `while (s->value == 0)` rather than decrementing `s->value` into negative numbers.
4. **`sem_down` calls `release(&s->lock)` BEFORE `sleep()`. What would go wrong if it slept holding the spinlock?** Two fatal problems would occur:
    - *Deadlock:* If a waiting process went to sleep still holding `s->lock`, any process calling `sem_up` would spin forever on `acquire(&s->lock)` and could never increment `s->value` or call `wakeup(s)`, so the sleeper could never wake up.
    - *Kernel panic / disabled interrupts:* In xv6, `acquire()` disables interrupts on the CPU core, and `sched()` panics (`panic("sched locks")`) if a process attempts to context-switch while holding any spinlock other than its own `p->lock`.

CONCLUSIONS

Implementing counting semaphores in the xv6 RISC-V kernel connected user-space synchronization semantics directly to their kernel implementation:

- In **Part 1**, adding system calls 24–27 across `kernel/syscall.h`, `user/user.h`, and `user/usys.pl` demonstrated how user calls trap into the kernel and why `semtest` fails ordering before the kernel handlers exist.
- In **Part 2**, `kernel/sem.c` showed that a kernel semaphore is simply an integer counter, a spinlock protecting it, and two-step kernel sleep/wakeup (`sleep_prepare(s)`, `release(&s->lock)`, `sleep()` and `wakeup(s)`) so waiting processes consume zero CPU cycles.
- In **Part 3**, `mix` and `mix lock` demonstrated multi-core interleaving across 31 single-byte `write()` system calls per line and proved that wrapping the loop in `sem_down(3)` / `sem_up(3)` turns the entire 31-syscall line into an indivisible critical section.